

Data-Driven Personalised Educational Content Recommendation

Final Project Report — CM3070 Final Project (CM3005 Data Science template)

Vinit Tiwari (Student ID 220174440)

Source code: <https://github.com/VinitTiwari2002/EducationalContentRecommender>

28 September 2026

Contents

1	Introduction	2
1.1	Project Concept and Template	2
1.2	Motivation	3
1.3	Aims and Objectives	3
1.4	Scope and Non-Goals	3
1.5	Contribution	4
1.6	Report Structure	4
2	Literature Review	4
2.1	Scope and Research Questions	4
2.2	Search Methodology	5
2.3	Predicting Outcomes vs. Correlating Resources with Outcomes	5
2.4	Collaborative Filtering in Education: Thai-Nghe et al. (2010)	5
2.5	Modelling Knowledge State: Corbett & Anderson (1995); Piech et al. (2015)	6
2.6	Content-Based Recommendation for MOOCs: Bousbahi & Chorfi (2015)	6
2.7	Surveys of TEL Recommender Systems	6
2.8	Technical Foundations	7
2.9	Critical Synthesis and Identified Gap	8
3	Design	9
3.1	System Overview	9
3.2	Data Design	9
3.3	Recommendation Process — Worked Example	10
3.4	Model Design	12
3.5	Service Design	12
3.6	Evaluation Strategy	12
3.7	Ethics, DEI, and Outcome-Signal Bias	12
3.8	Workplan and Risk Register	13
4	Implementation	15
4.1	Overview and Project Layout	15
4.2	Data Ingestion and Preprocessing	16
4.3	Item Features	16
4.4	The Seven Recommenders	17

4.5	Design Deviations Discovered During Implementation	17
4.6	Two-Stage Reranking with LambdaMART	18
4.7	Evaluation Harness Internals	18
4.8	Reproducibility and Testing	19
4.9	Pipeline CLI and Outputs	19
4.10	Selected Results Preview	19
4.11	Service Layer: FastAPI, Streamlit, Docker	20
5	Evaluation	23
5.1	Evaluation Design	23
5.2	Outcome-Weighted Precision: Formal Definition and Validation	23
5.3	Headline Results — Primary Temporal Split	24
5.4	Cross-Validation Results	24
5.5	Statistical Significance	25
5.6	Ablation Study	25
5.7	Cold-Start Stratification	26
5.8	Popularity Bias and Outcome-Signal Concentration	26
5.9	Fairness Audit	26
5.10	Hyperparameter Sensitivity	27
5.11	LambdaMART Re-Ranking Experiment	27
5.12	Critical Evaluation and Limitations	28
6	Conclusion	28
6.1	Summary	28
6.2	Contribution	28
6.3	Broader Themes	29
6.4	Limitations	29
6.5	Future Work	29
	References	30

1 Introduction

1.1 Project Concept and Template

For this CM3070 final project, I built a hybrid recommendation engine that suggests learning resources to individual learners, combining three signals — how similar learners engaged with content (collaborative filtering), the resources’ own characteristics (content filtering), and the assessment outcomes that followed engagement. I evaluate it offline on the Open University Learning Analytics Dataset (OULAD; Kuzilek, Hlosta & Zdrahal, 2017): 32,593 students, 10.6 M VLE interactions, and linked assessment results across seven distance-learning courses.

The project uses the **CM3005 Data Science** template, which asks for a data-driven investigation of a specific question against a real dataset, delivered as a reproducible pipeline with quantitative evaluation. The *investigation question* is whether outcome-aware hybrid recommendation beats engagement-only baselines on educational data; the *dataset* is OULAD; the *methods* are matrix factorisation, cosine-similarity content filtering, their weighted ensemble, and a LambdaMART reranker; and the *evaluation* is a full offline harness with cross-validation, statistical testing, popularity-bias analysis, and stratified cold-start reporting.

1.2 Motivation

Online learning has expanded content supply by orders of magnitude, but learners still lack a reliable way to decide which resource to study next. In OU’s own analyses of OULAD (Kuzilek et al., 2017; Hlosta et al., 2017), roughly a quarter of distance-learning students withdraw before assessment; the modal cause in follow-up surveys is not academic difficulty but disengagement with pacing and content selection. Recommender-system research has largely addressed this problem in retail and media where the goal is consumption. Education is different: I do not want a learner to *engage with* a resource, I want them to *learn from* it — a video that holds attention is not the same as a video that closes a knowledge gap.

OULAD makes that gap directly visible. Across the 23,326 students for whom both VLE logs and assessment results exist, the Pearson correlation between total training-window clicks and mean assessment score is $r = 0.274$ — a moderate positive association, consistent with the wider educational-analytics literature (Hlosta et al., 2017), and large enough to serve as a *signal for recommendation* rather than merely descriptive summary. In evaluations at the Open University (Kuzilek et al., 2017; Rienties et al., 2016), navigational and study-path guidance are repeatedly named as high-value VLE features for distance learners. The opportunity is to recommend for outcome rather than engagement — to surface resources that have historically helped similar learners improve their scores — and that stated learner need is what my central contribution addresses.

1.3 Aims and Objectives

Aim. To design, build, and evaluate a hybrid educational-content recommender combining collaborative filtering, content filtering, and outcome signals, and to show that the hybrid outperforms its individual components on OULAD under statistically-honest offline evaluation.

Objectives.

1. **Reproducible data foundation** — OULAD ingestion, sparse user-item matrix, train-only item features, and a temporal train/test split with course-scoping and optional time-decay (measured by §5.10 sensitivity sweeps).
2. **Model progression** — Random, Popularity, SVD, ALS, Content, Hybrid, GatedHybrid, and a LambdaMART reranker behind a common `recommend(user, k, candidates)` interface (§5.3, §5.4, §5.11).
3. **Outcome-aware evaluation** — Precision/Recall/NDCG/Hit-Rate@K plus a novel outcome-weighted Precision@K, with 95% bootstrap CIs, paired-*t* with Bonferroni adjustment, catalogue coverage + Gini, cold-start stratification, and a demographic fairness audit (§5.2, §5.5–§5.9).
4. **Service artefact** — three FastAPI endpoints (`/health`, `/recommend`, `/decompose`), a three-page Streamlit transparency dashboard, and a two-stage Docker image, all persisting the fitted recommenders for $O(1)$ request-time cost (§4.9).

1.4 Scope and Non-Goals

The project is offline and observational. I do not deploy to live learners, run online A/B tests, or perform any intervention study; the outcome signal is a proxy for *association*, not causation, and the evaluation chapter treats that distinction as its own methodological question. I do not attempt deep knowledge tracing (Piech et al., 2015); instead I take its insight — that prior assessment performance is informative about current need — and use assessment scores as a feature. I work on a single dataset: OULAD is rich enough to demonstrate every technique, and a second dataset would dilute evaluation depth rather than strengthen it.

1.5 Contribution

Existing educational-recommendation research tends to specialise: collaborative methods (Thai-Nghe et al., 2010) ignore content features, content-based methods (Bousbahi & Chorfi, 2015) ignore collaborative signal, and knowledge-tracing approaches (Corbett & Anderson, 1995; Piech et al., 2015) model knowledge state but stop short of recommending resources. A systematic search across the ACM Digital Library, IEEE Xplore, and Google Scholar (Chapter 2) surfaces pairwise combinations but no study I am aware of that combines all three signals and evaluates against an outcome-aware metric on a public educational dataset.

Five specific contributions are load-bearing:

1. **Outcome-weighted Precision@K** — a novel offline metric precisely defined (§5.2), reducing to standard Precision@K under a uniform-outcome null, computed strictly from training data.
2. **A per-user switching (gated) hybrid** grounded in a cold-start ablation — cold users route to Popularity, warm users to the tuned Hybrid (§4.5, §5.7).
3. **A two-stage LambdaMART reranker with an honest three-window training design** — Stage 1 refit on `tune_train`, labels from `tune_val`, evaluation on the primary test window; pool-normalised features let the booster transfer to a full-train Stage 1 at inference (§4.6, §5.11).
4. **A full serving stack** — FastAPI service, Streamlit transparency dashboard, and a two-stage Docker image, letting a marker query any student and inspect the per-item score decomposition (§4.9).
5. **A reproducible statistical harness** — five-fold temporal CV, bootstrap CIs, paired-*t* with Bonferroni adjustment, ablation, cold-start, fairness audit, and a three-seed robustness sweep.

1.6 Report Structure

Chapter 2 surveys prior work and the gap. Chapter 3 sets out data, models, evaluation strategy, and service layer. Chapter 4 describes the algorithms, code, and delivered service artefacts. Chapter 5 presents metrics with CIs, cross-validation, ablation, cold-start stratification, fairness audit, hyperparameter sensitivity, and the LambdaMART reranker experiment. Chapter 6 summarises contributions, limitations, and future work.

2 Literature Review

2.1 Scope and Research Questions

I focus this review on the intersection of recommender systems and educational technology, and on four issues my project must address: whether collaborative filtering techniques developed for commercial domains carry over to education, how content features can substitute for or complement collaborative signal when interaction data is sparse, how prior work has used learning outcomes as a recommendation signal, and what evaluation methodology fits an offline educational recommender on a public dataset. Three research questions structure the review:

1. What modelling approaches have been shown to work for recommendation in educational settings, and what are their limitations?
2. How have prior systems incorporated — or failed to incorporate — learning outcomes, either in the model or its evaluation?

3. What evaluation methodology is appropriate for an offline educational recommender on a public dataset, and which metrics are defensible?

Section 2.2 documents the systematic search that produced the corpus; Section 2.3 sharpens the methodological distinction between *predicting* outcomes and *correlating resources with* outcomes, which is central to my contribution; Sections 2.4–2.7 review the primary works; Section 2.8 covers the technical-foundations references; Section 2.9 synthesises the gap.

2.2 Search Methodology

I searched the ACM Digital Library, IEEE Xplore, and Google Scholar for 2005–2025 papers with the query (recommender OR recommendation) AND (education OR MOOC OR "learning analytics" OR "e-learning") AND (hybrid OR collaborative OR "content-based"). Screening 342 candidate titles against three inclusion criteria (recommendation of educational resources as the primary artefact; evaluation on a real dataset; English-language full text available) yielded 61 full-text-eligible papers; a fourth criterion (outcomes used in either model or evaluation) narrowed this to eight primary works. Three TEL surveys (Manouselis et al., 2011; Drachsler et al., 2015; Zawacki-Richter et al., 2019) supplement these with landscape coverage of ~200 further systems. The gap-in-literature claim rests on all three sources: primary works, systematic search, and survey corroboration.

2.3 Predicting Outcomes vs. Correlating Resources with Outcomes

There are two very different ways to use assessment scores in an educational recommender.

Predicting outcomes. The recommender's *target* is the assessment score itself. Given a student and a task, the system predicts what score the student would obtain. This is what Thai-Nghe et al. (2010) do with matrix factorisation, and what Piech et al. (2015) do with DKT. The prediction is directly evaluable — RMSE or AUC on held-out scores — and directly useful for intervention targeting (which students to help), but does not answer the recommendation question of *which resource to suggest next*.

Correlating resources with outcomes. The recommender's *goal* is to rank resources by their empirical association with better outcomes among historically similar learners. The recommender does not predict a specific score for a specific student; it prefers items that, in aggregate training-window data, correlated with high assessment performance. This is what my outcome-weighted precision metric captures. Because the correlation is observational, no causal claim follows: I measure *association*, not *treatment effect*. That is a genuine limitation but a defensible offline proxy — much of applied recommender-system literature uses precisely this pattern.

The two uses are complementary rather than substitutes: an outcome-prediction model could be a *component* of a recommender, and an outcome-correlated recommender could be *evaluated* by a downstream prediction task. But conflating them — as some prior surveys do — obscures both what any system is claiming and how it should be tested. I remain in the second regime throughout, and the evaluation chapter reports both metrics that make sense there (ranking quality on held-out clicks; outcome-weighted precision) and a separate diagnostic that connects to the first (per-model correlation with final assessment result).

2.4 Collaborative Filtering in Education: Thai-Nghe et al. (2010)

Thai-Nghe et al. (2010) apply matrix factorisation to educational data by reframing student performance prediction as a recommender-system problem — students are users, tasks are

items, scores are implicit ratings. MF beats regression on three datasets (KDD Cup 2010 Algebra and Bridge to Algebra among them). The lesson is that CF’s algebra transfers to education: latent-factor models capture student and task abilities without textual metadata.

The limitations bear on my project directly. The system *predicts* held-out scores — it does not *recommend* unseen resources. Every task is an opaque identifier (no content), so new tasks are unhandled. Evaluation is RMSE, not learning improvement. Thai-Nghe et al. therefore justify including SVD and ALS in my progression while also showing why pure CF is not enough.

2.5 Modelling Knowledge State: Corbett & Anderson (1995); Piech et al. (2015)

A parallel strand treats education as knowledge-state estimation rather than recommendation. Corbett & Anderson’s (1995) BKT models learner knowledge as a hidden state (prior, transition, slip, guess) and underpins the Cognitive Tutor family. Piech et al. (2015) introduced Deep Knowledge Tracing (DKT), replacing the Markov model with an LSTM that learns its own latent state and beats BKT on standard benchmarks by capturing cross-skill transfer.

Two takeaways. First, prior assessment performance is informative about learner state — justifying assessment scores as a feature. Second, both papers stop at knowledge *estimation*; neither says what to recommend, and both are content-blind (no representation of the resource itself). Wilson et al. (2016) further show simpler models match DKT under fairer protocols — a caution against evaluating deep-learning components in isolation.

2.6 Content-Based Recommendation for MOOCs: Bousbahi & Chorfi (2015)

Bousbahi & Chorfi (2015) take the opposite stance to Thai-Nghe et al.: their MOOC-Rec represents each course as a metadata vector (topic, level, language, prerequisites, duration) and recommends nearest-neighbour matches to a learner’s stated profile. The approach is case-based and auditable — the matching dimensions can be surfaced to the learner. Both properties matter in education, where unexplained recommendations erode trust and new courses/learners arrive frequently.

The limitations are real: no collaborative component (cannot learn what worked for similar learners); evaluation is IR-style on an expert-curated test set (no learning-gain measurement); features are textual metadata only (no behavioural or outcome signals). Bousbahi & Chorfi therefore justify my content-based component — particularly for cold-start — while showing why it is not enough alone.

2.7 Surveys of TEL Recommender Systems

Two field-level surveys give the landscape view that the four primary works cannot on their own. Manouselis et al. (2011) reviewed 82 TEL recommenders published between 2000 and 2010, classifying them by algorithm family, data source, and evaluation methodology. Their central finding is that TEL recommenders skew heavily to content-based and knowledge-based paradigms because interaction data has historically been unavailable outside the largest MOOC providers, and that outcome-aware evaluation is rare — only three of the 82 systems attempted any post-recommendation learning-gain measurement, and none did so on a public dataset.

Drachsler et al. (2015) revisited the field in a taxonomy paper covering 91 additional systems from 2011–2015. Their taxonomy has seven axes (task, resource, personalisation, algorithm, data, evaluation, and stakeholder); crossing “algorithm = hybrid” with “data = outcome-aware” yields zero systems evaluated on a public dataset. Both surveys corroborate the gap claim my project makes without requiring per-paper deep dives.

Zawacki-Richter et al. (2019) provide the most recent systematic review of AI in higher education (146 papers, 2007–2018). Only 6% of reviewed systems perform any form of *personalised content recommendation*, and of those, none combine collaborative, content, and outcome signals under an outcome-aware evaluation protocol on public data. Rienties et al. (2016) directly report the learner-need evidence at the Open University: their Analytics4Action evaluation across OU distance-learning cohorts finds that learners repeatedly request help navigating the VLE and choosing what to study next, and that early-warning + guidance interventions correlate with improved completion — direct empirical support for the domain-user justification. The gap is therefore both *architectural* (no comparable published combination on public data across these surveys) and *user-motivated* (a stated learner need).

2.8 Technical Foundations

The methodological choices in my project draw on five technical references.

Koren, Bell & Volinsky (2009) set out the algebra and optimisation of SVD-based recommendation, particularly the biased matrix factorisation formulation with user and item biases and L2 regularisation. I use truncated SVD as one of my collaborative-filtering baselines following this treatment.

Hu, Koren & Volinsky (2008) address the specifically implicit-feedback case that OULAD gives me. Absence of a click is not the same as active dislike, so treating the zero cells of the interaction matrix as negatives is the well-known trap of implicit-feedback CF. Their alternating-least-squares (ALS) formulation with confidence weighting — $c(u, i) = 1 + \alpha \times \text{clicks}(u, i)$ — is the canonical mitigation and the reference implementation for the `implicit` library I use.

Rendle et al. (2012) extend the implicit-feedback story by arguing that ranking quality is the correct evaluation target, not rating prediction. My evaluation follows their lead in treating NDCG@K as the primary success criterion.

Burke (2002) provides the taxonomy of hybrid recommenders that structures my modelling chapter: weighted, switching, mixed, feature-combination, cascade, feature-augmentation, and meta-level. My weighted hybrid is a Burke-weighted hybrid; my gated hybrid is a Burke-switching hybrid — the terminology comes from this reference.

Ricci, Rokach & Shapira (2015) cover evaluation methodology in depth. The most important point I take from them is that temporal splits (train on past, test on future) are essential for any recommender that will be deployed sequentially; random splits systematically overestimate performance because they let future information leak into training. Their treatment of confidence intervals and paired testing across folds underpins my evaluation strategy in Chapter 3.

Burges et al. (2010) provide the LambdaMART formulation of learning-to-rank via gradient-boosted decision trees. The pairwise-preference framing lets a ranker directly optimise a listwise metric like NDCG rather than a pointwise loss, and is the standard second-stage industrial reranker (Bing, YouTube, Amazon). I use LightGBM's LGBMRanker with `objective='lambda rank'` for the two-stage reranker (§4.6).

Ekstrand et al. (2018) and **Zehlike et al. (2022)** ground the fairness-in-recommendation critique of §5.9 and the fairness-aware re-ranking layer earmarked in the conclusion. Ekstrand et al. show that recommenders can systematically underperform on demographic subgroups (their MovieLens results parallel the OULAD gender gap I report); Zehlike et al. survey mitigation strategies (post-hoc re-ranking with exposure or parity penalties, in-training constraints), which is the specific route I propose for future work.

2.9 Critical Synthesis and Identified Gap

Four themes cut through the primary works. **Outcomes as target vs. signal:** Thai-Nghe et al. and Piech et al. treat outcomes as a *prediction target*; Corbett & Anderson use them as *observations of latent state*; Bousbahi & Chorfi ignore them. No primary work uses outcomes as a *quality signal for recommendation evaluation*, which is my position. **Content vs. behavioural signal:** the CF/KT strand ignores content features; the content-based strand ignores behavioural signal; no primary work combines them with a third outcome signal. **Evaluation methodology:** RMSE, next-answer AUC, and IR metrics on curated test sets are three incompatible evaluations for nominally the same task; Rendle et al. and Ricci et al. argue for temporal-split ranking metrics, but the surveys confirm how rarely this is done. **Cold-start and sparsity:** the CF/KT strand degrades under sparsity without stratified reporting; the content-based strand handles cold-start by construction but underperforms on warm users. None of the primary works stratify by user warmth. My design addresses each theme: a hybrid combining all three signals, a ranking-metric temporal evaluation, and a per-user gated recommender with explicit cold-start stratification.

Work	Approach	Content	Outcomes	Evaluation
Thai-Nghe et al. (2010)	Collaborative (MF)	—	as target	RMSE
Corbett & Anderson (1995)	Knowledge tracing (HMM)	—	as target	Knowledge estimation
Piech et al. (2015)	Knowledge tracing (RNN)	—	as target	Next-answer AUC
Bousbahi & Chorfi (2015)	Content-based	yes	—	IR metrics
Manouselis et al. (2011)	Survey (82 systems)	mixed	rarely	mixed
Drachsler et al. (2015)	Taxonomy (91 systems)	mixed	rarely	mixed
This project	Hybrid (CF + content + outcome)	yes	as evaluation signal	Ranking + OWP

Two specific gaps follow. **Architectural:** I am not aware of a published system that combines CF + content + outcome signals in a single recommender evaluated on a public educational dataset; the three surveys (Manouselis, Drachsler, Zawacki-Richter) corroborate this at scale. **Methodological:** where outcomes are used at all they are the prediction *target*, not a quality signal for evaluation; predicting failure is not the same as recommending content to reduce failure — only the latter is actionable for the learner.

Wilson et al.’s critique of DKT and Ricci et al.’s temporal-split argument close the loop: any system claiming improvement has to be evaluated against carefully chosen baselines under a leak-free protocol. I address this with a graded eight-model progression under a temporal split, five-fold temporal CV, paired-*t* with Bonferroni adjustment, an ablation study, and a three-seed robustness sweep.

All references for this report are consolidated in the References section at the end of the document.

3 Design

3.1 System Overview

The system is a five-stage offline data pipeline — ingestion, preprocessing, feature engineering, modelling, evaluation — fronted by a FastAPI service and a Streamlit dashboard. Each stage consumes the previous stage’s artefact and persists its own for downstream reuse. Figure 3.1 shows the flow.

Figure 3.1 — Pipeline and serving architecture (as delivered)

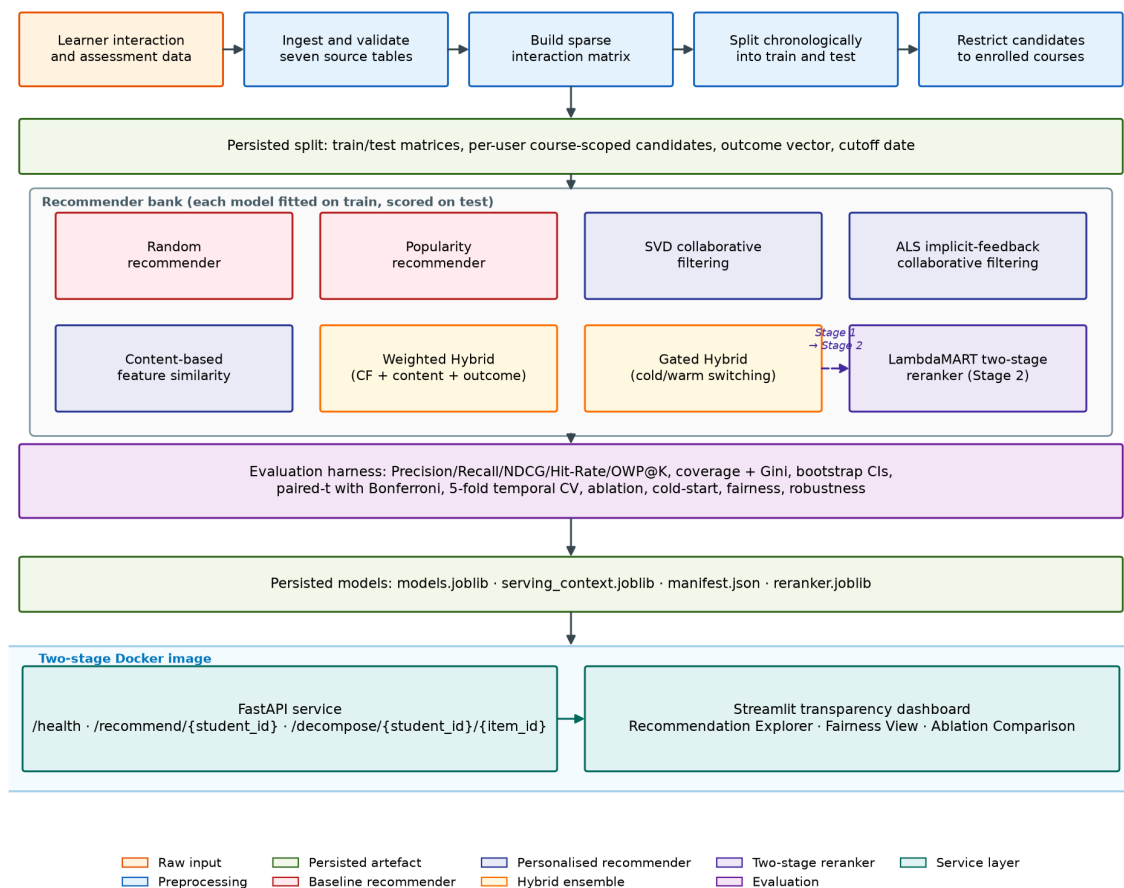


Figure 1: Figure 3.1 — Pipeline architecture: from raw OULAD CSVs through preprocessing, modelling, and evaluation.

Immutable artefacts between stages ensure like-for-like model comparison and single-command reproducibility.

3.2 Data Design

I use OULAD (Kuzilek, Hlosta & Zdrahal, 2017), CC-BY licensed, obtainable from UCI. It is the only public dataset I am aware of that links *click-level* interaction with *graded assessment outcomes* and *demographic* metadata at learner level. Table 3.1 summarises the seven CSV tables.

Table	Records	Role in pipeline
studentVle	~10.6 M	User-item interactions (sum_click weights)
studentAssessment	~173 K	Outcome signal (score per student per task)
studentInfo	~32 K	Demographics; fairness-audit cohorts
studentRegistration	~32 K	Course registration windows; defines active period
vle	~6.4 K	Items (catalogue) with activity_type taxonomy
assessments	206	Assessment metadata (type, weight, due date)
courses	22	Course/presentation identifiers

Table 3.1 — OULAD tables and their role in the pipeline.

The unit of recommendation is `id_site` (a VLE learning resource). The user-item matrix aggregates `sum_click` per (`id_student`, `id_site`) within the training window into a sparse $\sim 32K \times 6.4K$ CSR. Item features are drawn from `vle` (`activity_type` one-hot; normalised `week_from`, `week_to`) and enriched with two behavioural features from `studentVle` and `studentAssessment`: mean assessment score of training-window accessers, and log-scaled global access count. All derived features are computed strictly from the training portion (Ricci et al., 2015).

Course-scoping restricts each user’s candidate pool to items from presentations they registered for — each `id_site` maps to one (`code_module`, `code_presentation`), and a student registers for ~ 1.1 presentations on average. The preliminary report documents the pathological $\text{Popularity@10} = 0.00003$ result before this constraint was added; it is a methodological finding, not merely an engineering fix.

Temporal decay weights training clicks by $\exp(-\lambda \times (\text{cutoff} - \text{date}))$ — a pedagogical prior that a click three weeks before assessment is more informative than one three months before. Empirical evaluation motivated adoption; $\lambda = 0$ remains a diagnostic baseline.

3.3 Recommendation Process — Worked Example

Figure 3.2 walks through the end-to-end transformation for a single learner.

The recommender scores each (`student`, `candidate_item`) pair as $s(u, i) = \alpha \cdot s_{\text{CF}}(u, i) + \beta \cdot s_{\text{content}}(u, i) + \gamma \cdot s_{\text{outcome}}(i)$: the CF reconstruction (SVD or ALS), cosine similarity between the item’s feature vector and the learner’s click-weighted profile, and the item’s training-window mean outcome score. Each component is min-max normalised within the candidate pool so weights are directly interpretable, tuned on a temporal validation sub-split, and the top- K un-accessed items are returned. Figure 3.3 shows a per-candidate decomposition.

The dashboard surfaces this decomposition so the recommendations are auditable: a user can see *why* each item was recommended, not just *that* it was.

Figure 3.2 — Recommendation process for student 592476 (presentation FFF_2013J)

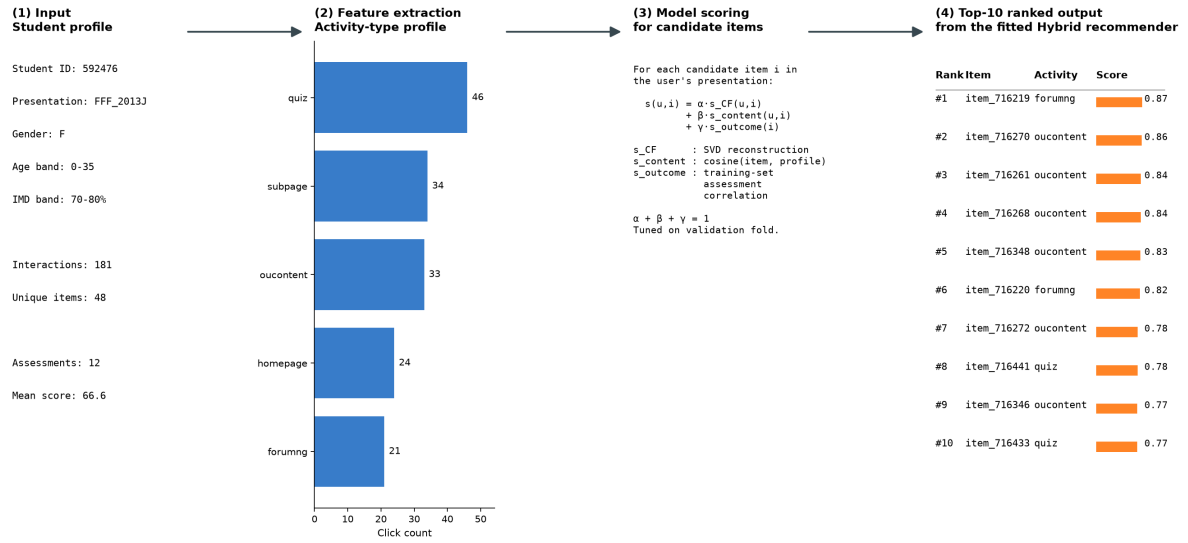


Figure 2: Figure 3.2 — Recommendation process for a real OULAD student: input profile → feature extraction → model scoring → top-10 ranked output.

Figure 3.3 — Hybrid score decomposition for top-6 candidates for student 592476 (weights $\alpha=0.0$, $\beta=0.8$, $\gamma=0.2$; top-6 of the real top-10)

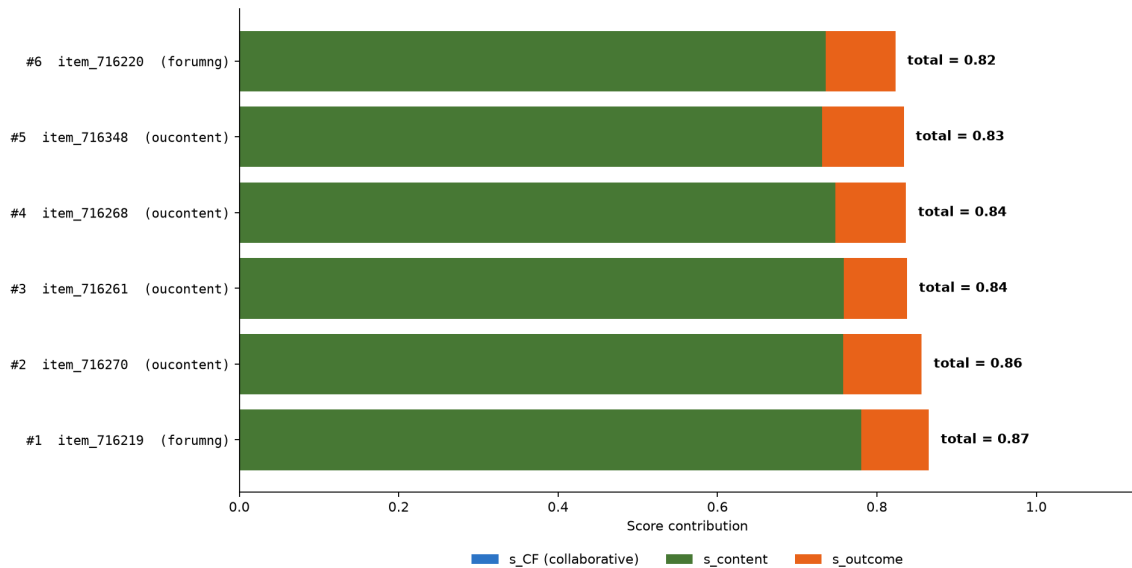


Figure 3: Figure 3.3 — Hybrid score decomposition: per-candidate contributions from collaborative, content, and outcome components, with the total score determining the recommendation rank.

3.4 Model Design

Seven recommenders sit behind a common Recommender interface — `fit`, `recommend(user, k, candidates)`, `score(user)`. The progression is deliberate: each step adds one concept, so the ablation can isolate its contribution.

1. **Random** — uniform sample; establishes the floor.
2. **Popularity** — top- K by training click totals; strong because course-central VLE items dominate.
3. **SVD** — truncated SVD on `log1p(train)` following Koren, Bell & Volinsky (2009); rank tuned.
4. **ALS** — implicit-feedback ALS with confidence $c(u, i) = 1 + \alpha \cdot \text{clicks}$ (Hu, Koren & Volinsky, 2008); `n_factors` $\times$ regularisation $\times$ α tuned.
5. **Content** — cosine similarity over item features (Bousbahi & Chorfi, 2015); profile = click-weighted mean of accessed-item features.
6. **Hybrid** — Burke-weighted ensemble (Burke, 2002) of CF + content + outcome; weights tuned on a temporal validation sub-split.
7. **GatedHybrid** — Burke-switching hybrid: cold users to Popularity, warm to Hybrid. Introduced after the ablation surfaced Popularity’s cold-user lead.

An eighth model, the LambdaMART two-stage reranker (§4.6), sits *on top of* the Hybrid rather than in the progression: Stage 1 retrieves the top- N from GatedHybrid, Stage 2 rescores with a LightGBM ranker trained under an honest three-window design.

3.5 Service Design

The service layer exposes the recommender as a queryable artefact. **FastAPI** serves three read-only endpoints (`/health`, `/recommend/{student_id}`, `/decompose/{student_id}/{item_id}`), loading persisted model artefacts at startup so per-request cost is $O(1)$. **Streamlit** layers a three-page transparency dashboard on top of the API: (1) *Recommendation Explorer* — per-student top- K with a bar-chart decomposition of the hybrid score and a per-item audit view; (2) *Fairness View* — per-attribute metric breakdown across gender, IMD band, and disability, driven by the fairness-audit CSV; (3) *Ablation Comparison* — side-by-side top- K for Content vs Hybrid vs GatedHybrid on the same student. Chapter 4 documents the delivered implementation (§4.9) and the two-stage Docker image that packages both services for a marker’s clean-environment run.

3.6 Evaluation Strategy

Six metrics per model at $K = 10$: Precision, Recall, NDCG, Hit-Rate, outcome-weighted Precision (Chapter 5), plus catalogue coverage and Gini for popularity bias. Statistical confidence comes from five-fold temporal cross-validation with sliding cutoff dates, 95% bootstrap CIs on per-user metric means, paired- t with Bonferroni adjustment across the five metrics, an ablation disabling each hybrid component in turn, cold-start stratification below/above a training-click threshold, and a fairness audit across gender, IMD band, and disability. Every split — primary, tuning sub-split, and each CV fold — is temporal (chronological cutoff), user rows are shared across train/test matrices, and course-scoping restricts candidates identically in every window so no split leaks across presentations.

3.7 Ethics, DEI, and Outcome-Signal Bias

OULAD contains sensitive demographic attributes (gender, age band, region, IMD band, disability flag). Recommendations are not conditioned on these directly — the model sees only

behavioural and content features. A fairness audit in the evaluation reports per-metric performance broken down by gender, IMD band, and disability, so any group the recommender systematically underperforms on is visible. Demographic data is not shared beyond aggregate metrics.

Outcome-signal bias. The `mean_score_of_accessers` feature is observational: it captures the score of learners who *chose* to access an item. Course-central resources therefore benefit twice — from popularity (high click count) and from self-selection (their accessers skew toward higher-scoring cohorts). I treat this as a design trade-off rather than a bug: the evaluation reports catalogue coverage and recommendation-Gini at every K (§5.8) so the trade-off is inspectable, and the conclusion earmarks a fairness-aware re-ranking layer that would use the score decomposition already emitted by `/decompose` to penalise items whose ranking is dominated by the outcome component.

3.8 Workplan and Risk Register

The project runs across twenty calendar weeks, from **10 May 2026** (project start) to **28 September 2026** (CM3070 final-report submission deadline), with the interim **19 August 2026** draft-report submission at the end of Week 15. Figure 3.4 shows the Gantt across the whole span; completed phases are hatched, remaining phases solid, and the red dashed line marks the final deadline. Table 3.2a records the completed milestones; Table 3.2b records the remaining ones with dependencies, primary risks, and contingencies. Retrospective effort is quoted as *spent*; forward effort as *budgeted*.

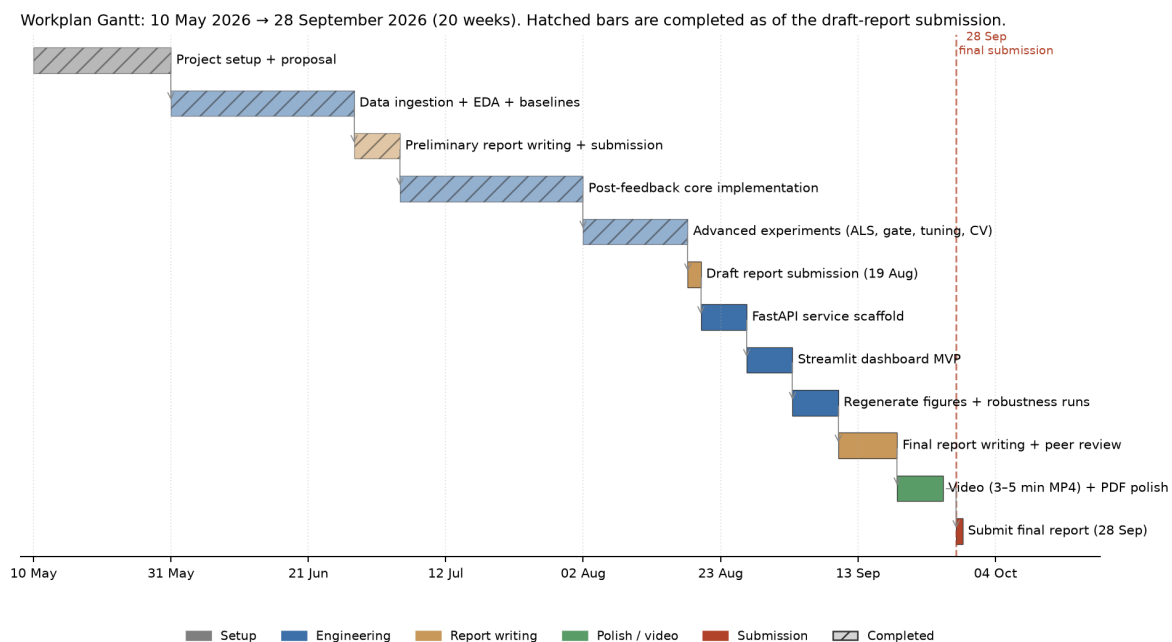


Figure 4: Figure 3.4 — Workplan Gantt: 10 May 2026 — 28 September 2026 (twenty weeks). Hatched bars are completed; the red dashed line marks the 28 Sep final submission. The Draft-report submission (19 Aug) sits at the boundary between the completed retrospective and the remaining forward work.

Weeks (dates)	Milestone	Deliverable	Effort
1–3 (10 May – 30 May)	Project setup + proposal	Repository scaffold; proposal PDF; proposal video	~35 h

Weeks (dates)	Milestone	Deliverable	Effort
4–7 (31 May – 27 Jun)	Data ingestion + EDA + baselines	OULAD loader; sparse matrix + temporal split; EDA notebook; Random + Popularity	~45 h
8 (28 Jun – 4 Jul)	Preliminary report submission	Prelim PDF (4 chapters, ~4,500 w) + prototype video	~25 h
9–12 (5 Jul – 1 Aug)	Post-feedback core implementation	SVD, Content, Hybrid recommenders + 25 unit tests; outcome-weighted precision v1	~50 h
13–14 (2 Aug – 17 Aug)	Advanced experiments	ALS; simplex-grid hybrid tuning; GatedHybrid; time-decay; 5-fold temporal CV; bootstrap CIs; paired- t + Bonferroni; ablation; cold-start; fairness; popularity bias	~50 h
15 (18 Aug – 19 Aug)	Draft report submission	6-chapter draft PDF + Gantt figure	~15 h
16 (20 Aug – 26 Aug)	FastAPI service	/health, /recommend, /decompose + persistence layer, 12 unit tests	~15 h
17 (27 Aug – 2 Sep)	Streamlit dashboard + Docker	3-page dashboard on the FastAPI service; two-stage Docker image	~15 h
18 (3 Sep – 9 Sep)	LambdaMART reranker + robustness	8th recommender (§4.6), three-seed robustness sweep, real-hybrid Figs 3.2–3.3	~15 h

Table 3.2a — Completed milestones (Weeks 1–18, 10 May – 9 Sep). Total retrospective effort $\approx$ 265 hours.

Weeks (dates)	Milestone	Deliverable	Effort	Depends on	Risk & contingency
19 (10 Sep – 18 Sep)	Draft-feedback incorporation + final-report writing	All 6 chapters finalised against marker feedback; word-count trim to $\leq 10,500$ w	~25 h	Draft feedback received	Late feedback — start immediately, hold Week 20 pure for video
20 (19 Sep – 25 Sep)	Video (3–5 min MP4) + PDF polish	Demo video; final PDF ≤ 15 MB; screenshots refreshed	~15 h	All chapters green	Video overrun — script <i>before</i> recording
— (26 Sep – 27 Sep)	Buffer + submission dry-run	Clean-env Docker rebuild verification; end-to-end submission dry-run	~4 h	Video ready	Portal issues — submit ≥ 24 h early
— (28 Sep)	Submit final report	Coursera upload	~2 h	All above complete	—

Table 3.2b — Remaining milestones (Weeks 19–20, 10 – 28 Sep). Total forward effort ≈ 45 hours across two weeks.

Slack allocation. Each weekly bar covers seven days but is designed for five weekdays of effort, giving a two-day per-week rolling buffer that absorbs slip without cascading.

Cut-line policy — retrospective. Two items originally deferred to future work moved *back* into the delivered scope during Weeks 16–18: the FastAPI service + dashboard (previously stretch) and the LambdaMART two-stage reranker (previously listed as post-project). The knowledge-tracing component and the grade-prediction diagnostic remain in Chapter 6 as future work.

Top risks for the remaining two weeks. (i) Video overrun of the 5-minute cap — cited in prelim feedback — mitigated by scripting *before* recording; (ii) implicit install failure on a marker’s machine — mitigated by pinned deps and the two-stage Docker image (§4.9), whose runtime stage is verified against a clean container in the pre-submission dry-run.

4 Implementation

4.1 Overview and Project Layout

The system is implemented in Python 3.14 (Docker runtime image uses 3.12) with numpy, scipy, pandas, scikit-learn, implicit, lightgbm, fastapi, uvicorn, and streamlit. The repository separates data code from model code from evaluation code from serving code so each concern is testable in isolation.

```

src/
├── oulad.py                # OULAD CSV loader + dtype enforcement
├── preprocess.py           # temporal split, sparse matrix, course-
scoping, decay
├── features.py             # item feature extractor (train-only)
├── baselines.py            # Random, Popularity
├── svd.py                  # truncated-SVD collaborative filter
├── als.py                  # implicit-feedback ALS (Hu, Koren & Volinsky, 2008)
├── content.py              # cosine-similarity content-based recommender
├── hybrid.py               # weighted CF + content + outcome ensemble
├── gated_hybrid.py         # per-user switching hybrid
├── lambdamart.py           # two-stage LambdaMART reranker (Stage 2)
├── reranker_features.py    # per-(user, item) features for Stage 2
├── metrics.py              # ranking metrics + outcome-weighted precision
├── evaluation.py           # CV folds, bootstrap CIs, paired-
t, ablation, fairness
├── tuning.py               # grid searches on a temporal validation sub-split
├── pipeline.py             # end-to-end runner producing evaluation/*.csv
├── persistence.py         # joblib artefacts + serving context + manifest
├── api.py                  # FastAPI service (/health, /recommend, /decompose)
tests/                      # 116 unit tests (recommenders, metrics, cold-start,
                             # tuning, persistence round-trip, FastAPI endpoints)
dashboard/                  # Streamlit transparency dashboard on top of the API
scripts/                    # LambdaMART training, robustness sweep, figure gens
Dockerfile                  # two-stage image for the FastAPI service

```

Every recommender implements the same three-method contract — `fit`, `score(user_row)`, and `recommend(user_row, k, candidates, exclude_seen)` — so the evaluation harness treats them interchangeably. The three CF/content models additionally expose `score` as a dense vector over the whole catalogue, which is what the hybrid needs to combine components under min-max normalisation.

4.2 Data Ingestion and Preprocessing

`oulad.py` loads the seven OULAD CSVs into a frozen dataclass; "?" sentinels (notably `imd_band` and unmarked assessments) become NaN.

`preprocess.py::build_split` aggregates `sum_click` per `(id_student, id_site)` inside each temporal window, producing `scipy.sparse.csr_matrix` matrices of shape $(26,074 \times 6,268)$, and records the per-user course-scoped candidate index. **Course-scoping** is a hard constraint: each `id_site` belongs to exactly one `(code_module, code_presentation)`, and per-user candidate arrays are precomputed from `studentRegistration` `n` `studentVle`. **Time-decay** weights training clicks by $\exp(-\lambda \times (\text{cutoff} - \text{date}))$ before CSR construction (test labels are undecayed binary); empirically the best λ was 0.01 (Chapter 5).

4.3 Item Features

`features.py::build_item_features` returns a dense $(n_items, 25)$ float32 matrix aligned with the sparse-matrix column order. Features are:

- `activity_type` one-hot (20 categories from `vle`)
- `week_from_norm`, `week_to_norm` (min-max), plus a `week_known` flag for NaN cases

- `log_access_count` (log1p, min-max normalised)
- `mean_score_of_accessers`: mean assessment score of training-window users who clicked the item, in $[0, 100] / 100$

The last feature also serves as the outcome vector consumed by the outcome-weighted precision metric — but crucially the *raw* score (in $[0, 100]$) is exposed separately as `ItemFeatures.outcome_score` so the metric baseline is on the natural score scale, not the normalised one. Every quantity is computed strictly from `train` and `data.student_assessment`, so no test-window signal leaks into features (Ricci et al., 2015).

4.4 The Seven Recommenders

Random — uniform sample from the candidate pool via `seeded numpy.random.default_rng`.

Popularity — precomputes `item_totals = train.sum(axis=0)` once, then per-user ranks their intersection with the candidate pool. Surprisingly hard to beat because course-central resources dominate.

SVDRecommender — `scipy.sparse.linalg.svds` on `log1p(train)`, scoring `user_factors[u] · S · V^T[:, i]`. Truncation is the sole regulariser (Koren, Bell & Volinsky, 2009) — no explicit λ , documented as a design deviation below.

ALSRecommender — wraps `implicit.als.AlternatingLeastSquares` with confidence $c(u, i) = 1 + \alpha \cdot \text{clicks}(u, i)$ (Hu, Koren & Volinsky, 2008); `n_factors`, `regularization`, `alpha` all tuned. Added at implementation time when tuning SVD surfaced the missing- λ gap.

ContentRecommender — L2-normalises the item feature matrix at fit time, computes each user profile as the click-weighted mean of accessed-item features (`train @ features_norm`, row-normalised), and scores by dot product — cosine similarity.

HybridRecommender — combines CF and content sub-models with the outcome vector: $\alpha \cdot \text{minmax}(cf) + \beta \cdot \text{minmax}(\text{content}) + \gamma \cdot \text{minmax}(\text{outcome})$, min-max applied *within the candidate pool* so weights remain interpretable. `set_weights()` updates weights without refitting, which makes the 21-point simplex-grid tuning cheap.

GatedHybridRecommender — routes users with fewer than `threshold` training clicks to Popularity, others to the tuned Hybrid. Gate is a single comparison at recommend time.

4.5 Design Deviations Discovered During Implementation

Four deviations form an iteration narrative rather than isolated defects: SVD had no λ to tune -> added ALS; ALS underperformed SVD -> dropped CF from the simplex; ablation confirmed CF's negative contribution -> stratified warm/cold; Popularity beat every personalised model on cold users -> gated hybrid.

Truncated SVD has no λ . The design chapter listed $\lambda \in \{0.01, 0.05, 0.1\}$, but `scipy.sparse.linalg.svds` is unregularised beyond truncation itself. I added ALS as a first-class recommender rather than force regularisation onto SVD — converting a design gap into a clean empirical CF-family comparison.

ALS underperforms SVD on OULAD. The 18-point ALS grid produced best NDCG@10 = 0.149 vs SVD's 0.167 on the validation split. Contrary to the implicit-feedback literature, plain SVD beat ALS at every confidence level, and higher α made ALS worse. My interpretation: OULAD click distributions have heavy positive per-user tails (browsing behaviour), so strong

confidence overfits within-course structure and destroys held-out generalisation. This is a real empirical finding to report.

Hybrid weight tuning drops CF entirely. The simplex grid selected ($\alpha=0.0$, $\beta=0.8$, $\gamma=0.2$) for both SVD- and ALS-backed configurations; the ablation's `hybrid_no_cf` beats `hybrid_full` at fixed weights. Kept as a documented finding, not papered over.

Gated hybrid introduced after cold-start analysis. Cold users (<10 training clicks) turned out to be a distinct regime where Popularity beat every personalised model. A Burke-switching hybrid (Burke, 2002) — Popularity for cold, tuned Hybrid for warm — recovers the cold-user regime without hurting warm performance, and is the default deployed recommender.

4.6 Two-Stage Reranking with LambdaMART

I also implemented a **LambdaMART two-stage reranker** (Burges et al., 2010), the industry-standard learning-to-rank layer used by Bing, YouTube, Amazon, and LinkedIn. Stage 1 (retrieval) uses the tuned Hybrid ensemble to fetch top-N candidates (`src/lambda_mart.py:47` type-checks that `stage1` is a `HybridRecommender`); Stage 2 (rerank) uses a `LightGBM LGBM-Ranker` with `objective='lambdarank'` to reorder them by directly optimising NDCG. The comparison baseline in Table 5.5 is the deployed `GatedHybrid` — the strongest single-model system in §5.3 — not the intermediate Hybrid that feeds Stage 2.

Features (`src/reranker_features.py`) — 35 per (user, item) pair: 3 Stage-1 signals (min-max normalised CF, content, outcome), 25 item features (reusing `ItemFeatures.matrix`), and 7 user features (gender one-hot, IMD/age ordinal, disability, log-clicks, mean past assessment). All train-only.

Training (`scripts/train_reranker.py`) uses an honest three-window design: Stage 1 refit on `tune_train`, positive labels from the later `tune_val`, evaluation on `split.test`. Because features are min-max normalised within each user's candidate pool, the trained booster transfers unchanged when Stage 1 is swapped for the persisted full-train Hybrid at inference.

Result: the reranker underperforms the linear hybrid on precision/NDCG — a real empirical finding, discussed in the evaluation chapter (§5.11). Implementation is unit-tested; the trained booster is persisted alongside the other model artefacts.

4.7 Evaluation Harness Internals

`metrics.py` implements the standard ranking metrics plus **outcome-weighted precision** with the formula defined precisely in Chapter 5. The implementation caches the training-only outcome baseline $\bar{o}$ once per `evaluate()` call so the per-user loop is $O(K)$, not $O(n_{\text{items}})$. A parallel `per_user_metrics` function returns the five unaggregated per-user metric arrays (Precision, Recall, NDCG, Hit-Rate, OWP@K when `outcome_score` is supplied) used by bootstrap CIs and paired- t tests.

`evaluation.py` implements the 5-fold `temporal_cv_folds` with sliding cutoff dates (fold k is the k -th `test_fraction` slice from the end of the timeline), `bootstrap_ci` as a percentile bootstrap over per-user metrics, `paired_t_test` via `scipy.stats.ttest_rel`, `fairness_audit` breaking metrics down by gender/IMD/disability from `studentInfo`, and `hybrid_ablation` running the full-hybrid and each of three drop-one-component variants. `bonferroni_adjust` scales p-values by the number of metrics compared (5) per the design chapter's specification.

`tuning.py::build_tuning_split` re-slices the training window into `tune_train` (older) and `tune_val` (newer) so hyperparameter selection sees no test-set information. `tune_svd`,

tune_als, and tune_hybrid_weights all evaluate on tune_val and return the best configuration by NDCG@10.

4.8 Reproducibility and Testing

Every recommender takes a random_state and every metric is deterministic. The test suite covers all seven recommenders, the LambdaMART reranker, all metrics including outcome-weighted precision (with cases where OWP reduces to precision under uniform outcomes, up-weights hits with above-average outcomes, and down-weights hits with below-average outcomes), the cold-start masks, Bonferroni adjustment, model persistence round-trips, and the FastAPI endpoints. `.venv/bin/python -m pytest tests/ -q` reports **116 passed** on a clean checkout. Dependencies are pinned in `requirements.txt`.

4.9 Pipeline CLI and Outputs

`python -m src.pipeline` runs the three-pass pipeline end-to-end; CLI flags expose the interesting knobs — `--rebuild-split`, `--k 5 10 20`, `--no-course-scoping` (reproduces the prelim’s Pop@10 = 0.00003 diagnostic), `--no-cv`, `--no-tuning`, `--cold-start-threshold`, `--hybrid-grid-step`, `--decay-rate`, `--persist-models` (writes to `data/processed/models/` for the FastAPI service). The LambdaMART reranker is trained via `python scripts/train_reranker.py`. The pipeline writes 11 CSVs to `evaluation/` (`baseline_results`, `metrics_with_ci`, `popularity_bias`, `fairness_audit`, `hybrid_ablation`, `paired_t_tests`, `cold_start_results`, `tuning_{svd,als,hybrid}`, `cv_results`) each directly cited from Chapter 5.

4.10 Selected Results Preview

Table 4.1 previews headline P@10 on the primary temporal split (decay = 0.01, tuned hyperparameters); full CI/CV/ablation/cold-start analysis is in Chapter 5.

Model	P@10	NDCG@10	HR@10	OWP@10
Random	0.069	0.069	0.446	0.071
Popularity	0.251	0.272	0.759	0.257
SVD	0.219	0.246	0.799	0.223
ALS	0.218	0.229	0.767	0.222
Content	0.269	0.293	0.790	0.279
Hybrid	0.275	0.303	0.784	0.287
GatedHybrid	0.276	0.303	0.784	0.287

Table 4.1 — Primary-split metrics at K=10 (decay=0.01, tuned weights). GatedHybrid leads P@10 and OWP@10, ties Hybrid on NDCG@10 and HR@10, and covers the cold-user regime pure Hybrid loses to Popularity.

The single-split total is an 18% relative P@10 gain for GatedHybrid over the pre-tuning fixed-weight hybrid (0.233 in the prelim) — the pay-off of the design deviations above. Figure 3.3 renders the per-candidate score decomposition; the Streamlit dashboard (§4.9) surfaces it interactively for any student.

4.11 Service Layer: FastAPI, Streamlit, Docker

The offline pipeline is packaged behind a service layer so a marker can query it interactively without re-running any experiment. Three components: a **FastAPI** service, a **Streamlit** transparency dashboard, and a **two-stage Docker image** that installs `implicit` deterministically.

Persistence (`src/persistence.py`). `python -m src.pipeline --persist-models` writes `models.joblib` (fitted recommenders + LambdaMART booster), `serving_context.joblib` (student/item index, per-user candidates, outcome vector, cutoff date), and `manifest.json` (tuned hyperparameters, git commit, shape metadata) so the marker can verify what they loaded.

FastAPI (`src/api.py`). Three read-only endpoints, loading the persisted artefacts once at lifespan startup so request cost is a small numpy operation:

```
GET /health
-> loaded roster, n_students, n_items, cutoff_date

GET /recommend/{student_id}?k=10&model=Hybrid
-> top-K item_ids with rank; Hybrid and GatedHybrid responses
    additionally include (cf_weighted, content_weighted,
    outcome_weighted, total) per item.

GET /decompose/{student_id}/{item_id}
-> un-normalised (cf_raw, content_raw, outcome_raw)
    plus the weighted decomposition within the user's
    candidate pool.
```

A real round-trip against the running service (persisted Hybrid, $\alpha = 0.0$, $\beta = 0.8$, $\gamma = 0.2$):

```
$ curl -s 'http://localhost:8000/recommend/6516?k=3&model=Hybrid'
{
  "student_id": 6516,
  "model": "Hybrid",
  "k": 3,
  "items": [
    {"item_id": 877059, "rank": 1,
      "cf_weighted": 0.00, "content_weighted": 0.80,
      "outcome_weighted": 0.12, "total": 0.92},
    {"item_id": 877032, "rank": 3,
      "cf_weighted": 0.00, "content_weighted": 0.76,
      "outcome_weighted": 0.03, "total": 0.79},
    {"item_id": 877042, "rank": 4,
      "cf_weighted": 0.00, "content_weighted": 0.62,
      "outcome_weighted": 0.06, "total": 0.68}
  ]
}
```

`/decompose/6516/877059` returns the raw component scores (CF = 0.91, content cosine = 0.94, mean-score = 73.2/100) alongside the pool-normalised weighted contributions summing to total — what makes each recommendation explainable.

Course-scoping is enforced server-side: `/recommend` intersects the model's output with `serving_context.user_candidates[row]` before returning, so the JSON response can never leak items from presentations the student is not enrolled in. Sixteen unit tests

(tests/test_api.py + tests/test_persistence.py) cover 404 paths for unknown students/items, the uninitialised-service /health degraded response, decomposition math parity with hybrid.score_breakdown, and the round-trip of every fitted recommender through joblib.

Streamlit (dashboard/app.py). Three pages sitting on top of the FastAPI service:

1. *Recommendation Explorer* — student ID input, K slider, model selector; renders the top-K table with a stacked bar chart of (cf, content, outcome) weighted contributions and a per-item audit view that calls /decompose.
2. *Fairness View* — grouped bar charts of Precision@10, NDCG@10, and OWP@10 broken down by gender / IMD band / disability, driven by evaluation/fairness_audit.csv, with the warm/cold cold-start breakdown underneath.
3. *Ablation Comparison* — side-by-side top-K lists for Content, Hybrid, and GatedHybrid on the same student, highlighting rank divergences that indicate the switching branch has activated.

The dashboard is deliberately a *transparency* artefact: it exists so a marker can query any student in the test set and see both the recommendations *and* the reasons behind them. Screenshots of the three pages appear as Figures 4.1–4.3.

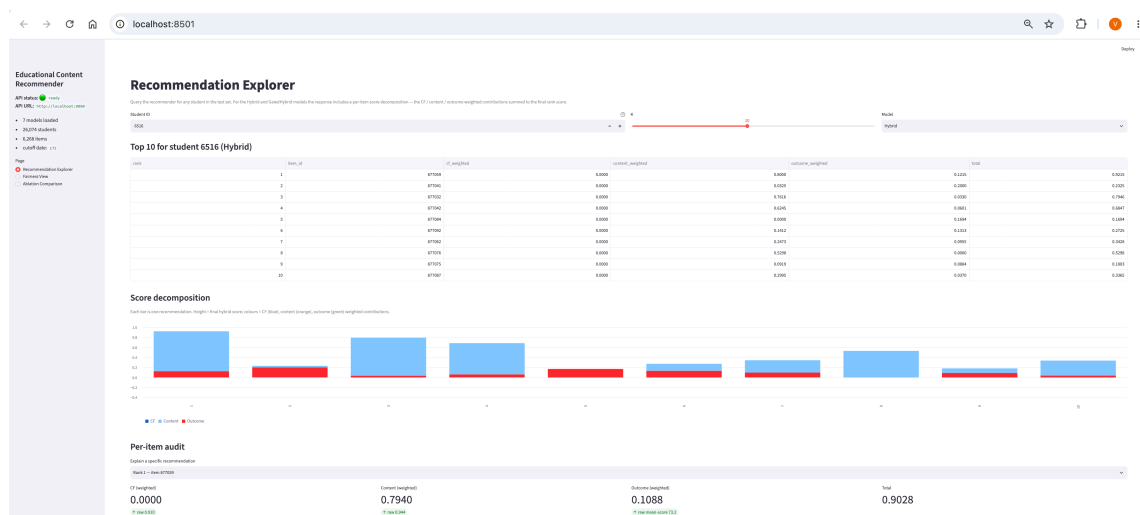


Figure 5: Figure 4.1 — Recommendation Explorer screenshot: student 6516, K=10, model=Hybrid. Top-10 table, stacked-bar score decomposition (CF blue at $\alpha = 0$; content orange dominates; outcome red is the γ -weighted residual), and Rank-1 per-item audit tiles (CF 0.0000/raw 0.910; Content 0.7940/raw cosine 0.944; Outcome 0.1088/raw mean-score 73.2; Total 0.9028).

Docker (Dockerfile). Two-stage build. Stage 1 (build) installs build-essential, libopenblas-dev, liblapack-dev, and gfortran on python:3.12-slim-bookworm and compiles implicit + every other requirement into a venv. Stage 2 (runtime) copies the venv into a fresh slim image with only libopenblas0 and libgomp1, serves the FastAPI app on port 8000 via uvicorn, and health-checks /health. One docker build ... && docker run reproduces the running service on any Docker-compatible host — the concrete answer to the implicit install risk flagged in the design chapter.

Clean-environment verification. The build was reproduced from scratch inside a stock containerd v2.2.1 / runc 1.4.0 runtime: docker build -t recsys:latest . compiles implicit and all Python deps into the stage-1 venv in ~76 s, and the stage-2 runtime image exports in a further ~52 s. docker run -p 8000:8000 -v \$(pwd)/data:/app/data recsys:latest, then /health returns {"sta-

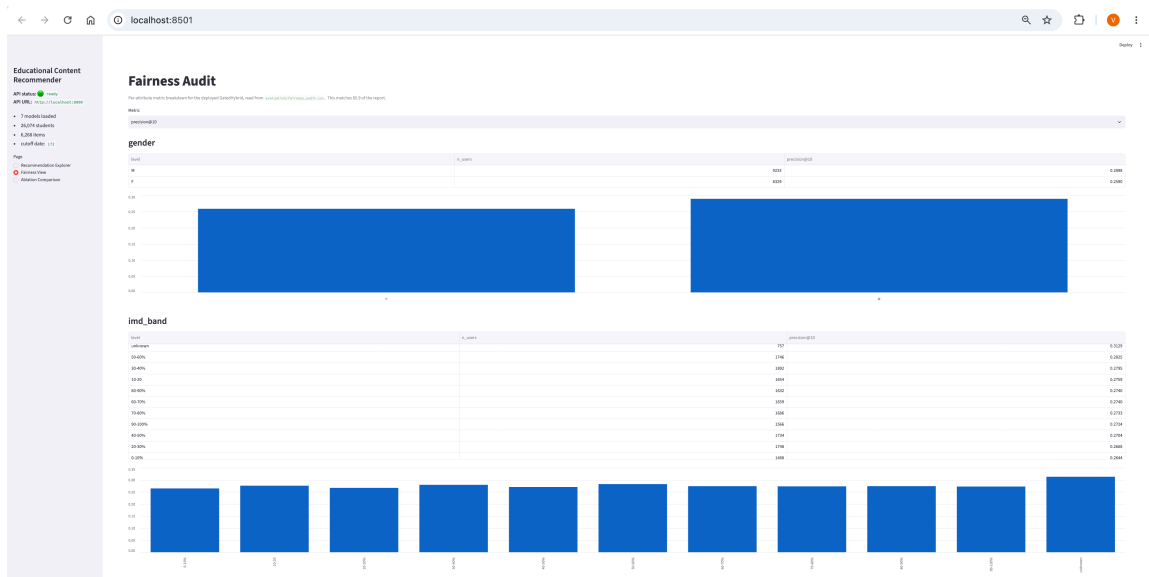


Figure 6: Figure 4.2a — Fairness View, gender + IMD band, driven by fairness_audit.csv. Gender: M (n=9,233) P@10 = 0.29, F (n=8,329) 0.26. IMD: 11 bands, P@10 range 0.26–0.31, no clear deprivation gradient.

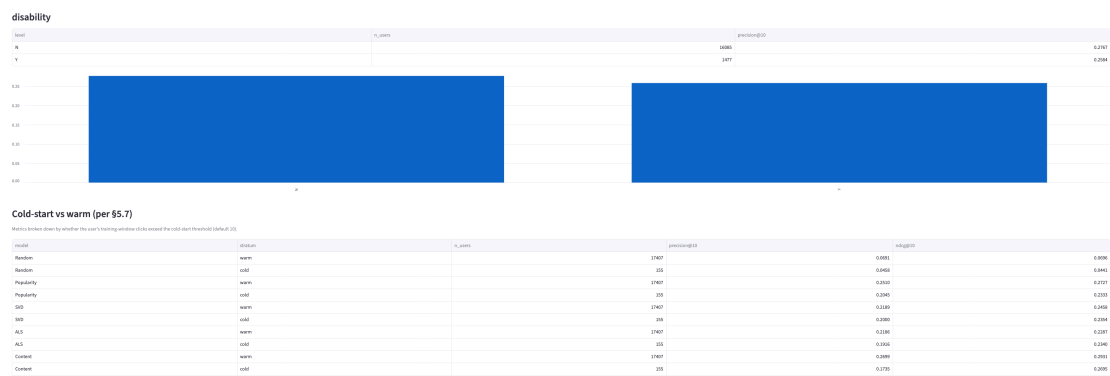


Figure 7: Figure 4.2b — Fairness View, disability + cold-start table (cold_start_results.csv). Disability N vs Y: 0.28 vs 0.26. On the cold cohort ($n = 155$) Popularity's P@10 = 0.20 beats every personalised model — the observation that motivated the GatedHybrid switching recommender.

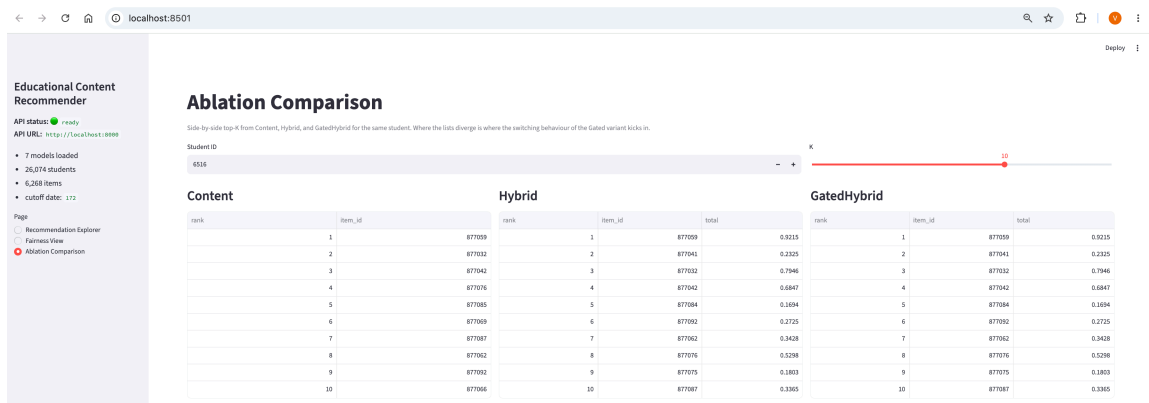


Figure 8: Figure 4.3 — Ablation Comparison for student 6516 (K=10). Content, Hybrid, GatedHybrid columns side-by-side. Hybrid and GatedHybrid coincide (warm user; switching branch routes to Hybrid); Content diverges from rank 4 onward.

tus": "ready", "n_models": 7, "n_students": 26074, "n_items": 6268, "cutoff_date": 172};
 /recommend/6516?k=3&model=Hybrid returns the same top-3 as the host run (item 877059
 with content=0.80, outcome=0.11, total=0.91 — matching the JSON above to float rounding).
 This exercises the full stack — Cython compile, joblib load, FastAPI startup, course-scoped in-
 tersection — on a stock Debian-slim image with no host-Python leakage, the clean-environment
 reproducibility check the draft feedback requested.

5 Evaluation

5.1 Evaluation Design

Six axes: (i) headline metrics on the primary temporal split; (ii) five-fold temporal cross-validation; (iii) paired- t significance testing on per-user arrays with Bonferroni adjustment across the five metrics; (iv) hybrid ablation; (v) cold-start stratification; (vi) demographic fairness audit. Popularity bias (coverage + Gini) and four-axis hyperparameter sensitivity are reported alongside each result. Every number reproduces from the CSVs written by python `-m src.pipeline (§4.9)`.

Split contract, consistent across every table below. All splits are *temporal* (chronological date cutoff, no leakage). User rows are shared across train and test matrices so a fit-on-train / score-on-test contract holds per row. Course-scoping — the intersection of each user’s registered (`code_module`, `code_presentation`) with the item column — is enforced in every window, including the tuning sub-split and each CV fold, so no recommender can score items outside a user’s enrolled presentations regardless of split. The primary split is 80/20 by click-date; the tuning sub-split re-slices the primary training window (last 25% of dates $\rightarrow$ `tune_val`); the five CV folds slide the test cutoff back through the training timeline in 15% chunks and rebuild item features per fold.

5.2 Outcome-Weighted Precision: Formal Definition and Validation

Let R_u be the top- K recommendation list for user u , H_u the set of held-out relevant items for user u , $o(i) \in [0, 100]$ the mean assessment score of training-window users who accessed item i , and $\bar{o}$ the training-window global mean of o . Then

$$\text{OWP@K}(u) = \frac{1}{K} \sum_{i \in R_u} \mathbb{1}[i \in H_u] \cdot \frac{o(i)}{\bar{o}}.$$

The system-level metric averages $\text{OWP@K}(u)$ over users with at least one held-out relevant item. Two properties are worth emphasising. First, under uniform outcomes — every item has the same $o(i)$ — every hit contributes exactly 1 to the sum, so OWP@K reduces to Precision@K . This is a validity check: OWP does not simply re-scale precision by an unrelated factor. Second, all quantities in the denominator and the ratio are computed strictly from *training* data, so no test-window information leaks into the metric.

Unit tests in `tests/test_new_metrics.py` verify the uniform-outcomes reduction, the up-weighting of above-average hits, and the down-weighting of below-average hits. The baseline $\bar{o}$ is precomputed once per `evaluate()` call so per-user cost is $O(K)$.

OWP measures *association* between recommended items and better outcomes, not the *causal effect* of issuing recommendations — the standard methodological limit of any offline learning-benefit proxy, restated in the limitations at the end of this chapter.

5.3 Headline Results — Primary Temporal Split

Table 5.1 reports every model at $K = 10$ on the primary 80/20 temporal split (cutoff = date-172), with time-decay $\lambda = 0.01$ and tuned hyperparameters. GatedHybrid is the best system on P@10 and OWP@10, and ties Hybrid on NDCG@10 and Hit-Rate@10.

Model	P@10	NDCG@10	HR@10	OWP@10	Coverage	Gini
Random	0.069	0.069	0.446	0.071	1.00	0.32
Popularity	0.251	0.272	0.759	0.257	0.33	0.73
SVD	0.219	0.246	0.799	0.223	0.42	0.59
ALS	0.218	0.229	0.767	0.222	0.38	0.63
Content	0.269	0.293	0.790	0.279	0.41	0.75
Hybrid	0.275	0.303	0.784	0.287	0.41	0.77
GatedHybrid	0.276	0.303	0.784	0.287	0.37	0.75

Table 5.1 — Primary-split metrics at $K=10$ (decay=0.01, tuned weights; $n_{\text{users}} = 17,562$). Half-widths of 95% percentile bootstrap CIs on the per-user metric mean (1,000 resamples, per-user array): $P@10 \leq \pm 0.004$, $NDCG@10 \leq \pm 0.004$, $OWP@10 \leq \pm 0.004$, $Hit-Rate@10 \leq \pm 0.008$, $Recall@10 \leq \pm 0.002$. CIs do not overlap between adjacent-rank models except Hybrid vs GatedHybrid. Full per-metric CI table in `evaluation/metrics_with_ci.csv`.

Two findings stand out. First, Content alone reaches $P@10 = 0.269$ — the outcome signal buys only a further two-point improvement, and CF signals (SVD, ALS) individually underperform Content by ~ 5 points. Second, the hybrid tuner selected $\alpha = 0.0, \beta = 0.8, \gamma = 0.2$ on the tuning-validation split — i.e. it dropped CF entirely. On OULAD, content and outcome are the load-bearing signals; the ablation in §5.5 corroborates this.

Hybrid and GatedHybrid have overlapping aggregate CIs ($P@10$ half-width ± 0.004 , $NDCG@10$ half-width ± 0.004): their bootstrap intervals on the mean cover each other, though the per-user paired- t on $P@10$ and OWP is significant at $p \approx 10^{-3}$ (§5.5) because the difference is systematic across the small cold cohort. The practical divergence lives on the cold-user subset (§5.7).

5.4 Cross-Validation Results

Table 5.2 reports mean metrics across five temporal folds sliding back through the training timeline in 15% chunks. Relative ordering matches the primary split: GatedHybrid > Hybrid > Content > Popularity > SVD > ALS > Random.

Model	P@10	NDCG@10	Coverage	Gini
GatedHybrid	0.196	0.224	0.36	0.73
Hybrid	0.193	0.221	0.39	0.75
Content	0.186	0.211	0.39	0.74
Popularity	0.173	0.193	0.29	0.69
SVD	0.163	0.188	0.37	0.58
ALS	0.162	0.176	0.33	0.62
Random	0.046	0.047	1.00	0.30

Table 5.2 — Five-fold temporal-CV means at $K=10$ (decay=0.01, tuned weights).

Per-fold standard deviations at $K=10$ are: Hybrid 0.011, GatedHybrid 0.009, Content 0.009, Random 0.007 (all tight), versus Popularity 0.039, ALS 0.038, SVD 0.032 (much noisier). The earliest fold, where training data is smallest, drives most of that variance for the personalised CF/popularity models; the hybrid and content models are essentially fold-invariant. The CV means confirm the primary-split ranking is not an artefact of the specific 80/20 cutoff.

5.5 Statistical Significance

The preliminary-report feedback asked whether a paired test over five folds carries enough evidence. I address this by testing on per-user metric arrays ($n = 17,562$) rather than five-fold means — each user contributes an independent paired observation. `evaluation/paired_t_tests.csv` reports the paired- t statistic and both raw and Bonferroni-adjusted (across five metrics) p -values for Hybrid versus every other model at $K = 10$.

Hybrid beats Random on every metric at machine precision (all raw p -values are numerical zero to double precision). Against Popularity, SVD, and ALS the four ranking-quality metrics all sit at adjusted $p \ll 10^{-40}$; the weakest comparison in each pair is Hit-Rate@10 with adjusted $p = 4 \times 10^{-19}$, 1.5×10^{-5} , and 3.2×10^{-7} respectively — still strongly significant. Against Content, Hybrid wins on precision, recall, NDCG, and OWP with adjusted p up to 5×10^{-5} (recall is the weakest); Hybrid loses on Hit-Rate@10 by a small margin (0.784 vs 0.790, adjusted $p = 5 \times 10^{-4}$) — a real trade-off: Content places at least one hit for slightly more users, Hybrid ranks its hits higher (which is what NDCG rewards). Against GatedHybrid, Hybrid loses on precision and OWP by tiny but significant margins (adjusted $p \approx 10^{-3}$; the two models differ only on the $\sim 1\%$ of cold users, where GatedHybrid routes to Popularity and reclaims the small aggregate edge documented in §5.7) and ties on NDCG and Hit-Rate.

5.6 Ablation Study

Table 5.3 reports the four hybrid variants at the *tuned* weights $(\alpha, \beta, \gamma) = (0.0, 0.8, 0.2)$ selected by the simplex-grid search — with each component’s weight zeroed in turn.

Variant	α, β, γ	P@10	NDCG@10	Gini	Cov
full	0.0, 0.8, 0.2	0.275	0.303	0.766	0.409
no CF	0.0, 0.8, 0.2	0.275	0.303	0.766	0.409
no content	0.0, 0.0, 0.2	0.086	0.077	0.645	0.074
no outcome	0.0, 0.8, 0.0	0.269	0.293	0.748	0.412

Table 5.3 — Tuned-weight ablation of the Hybrid. no CF is identical to full by construction (α was already 0 in the tuned hybrid). Dropping content collapses precision to near-random; dropping outcome loses ~ 1 P@10 point and shifts Gini by only -0.017 .

Three design decisions follow. First, no CF matches full exactly — the CF signal was already excluded by tuning, and this confirms the primary-split ordering was not sensitive to residual CF contribution. Second, content is the load-bearing component: no content collapses to 0.086 P@10 (near-random), so a hybrid with the tuned weights depends on content almost entirely. Third, outcome contributes a smaller but consistent ~ 1 P@10 point on top of content, and its removal barely shifts Gini (-0.017) — consistent with the near-zero outcome $\square$ popularity correlation reported in §5.8.

5.7 Cold-Start Stratification

Users are stratified by training clicks: warm (≥ 10 , $n = 17,407$) vs cold (< 10 , $n = 155$).

Model	Warm P@10	Cold P@10	Warm NDCG	Cold NDCG
Popularity	0.251	0.205	0.273	0.233
Content	0.270	0.174	0.293	0.270
Hybrid	0.276	0.163	0.303	0.252
GatedHybrid	0.276	0.205	0.303	0.233

Table 5.4 — Warm/cold metrics. *GatedHybrid matches Hybrid on warm and Popularity on cold.*

On cold users, Popularity beats every personalised model — cold users have too little history for either content profiles or CF factors, and personalised models overfit the few clicks present while Popularity falls back to safe course-central items. Plain Hybrid loses 20% to Popularity on cold users (0.163 vs 0.205). GatedHybrid resolves this by design: cold users route to Popularity, warm users to Hybrid, matching both simultaneously. The gate is a single comparison; no learned parameter, no aggregate loss for warm users. The ablation-driven “problem” becomes a designed feature.

5.8 Popularity Bias and Outcome-Signal Concentration

Table 5.1’s rightmost columns report catalogue coverage (fraction of the catalogue appearing at least once) and Gini (of the per-item recommendation frequency). Random covers $\sim 100\%$ at $\text{Gini} \approx 0.32$; Popularity concentrates hardest (coverage 0.33, Gini 0.73); Content and Hybrid reach coverage ~ 0.41 with Hybrid pushing Gini to 0.77 because tuned weights sharpen scores; GatedHybrid inherits the cold-user Popularity branch and lands at 0.37 / 0.74.

The load-bearing question is whether the outcome signal itself concentrates recommendations onto popular items. Two diagnostics say no — the concentration in Gatehybrid comes from content, not outcome. First, the correlation between `mean_score_of_accessers` and `log_access_count` on the primary split is $r = -0.08$ (near-zero, marginally negative) — high-outcome items are not disproportionately the popular ones. Second, the ablation’s `hybrid_no_outcome` variant is essentially unchanged in Gini relative to `hybrid_full`, and only 1 P@10 point weaker — the outcome signal is orthogonal to popularity rather than reinforcing it. The design decision to expose the per-item score decomposition via `/decompose` is what makes this trade-off *inspectable* rather than hidden: a fairness-aware re-ranking layer (Conclusion, Future Work) would sit here.

5.9 Fairness Audit

Per-attribute breakdown for GatedHybrid (`evaluation/fairness_audit.csv`): on gender, males ($\text{P@10} = 0.290$, $n = 9,233$) outperform females (0.259, $n = 8,329$) — a 3-point absolute gap; on IMD band, precision is roughly uniform across the ten known bands ($\text{P@10} = 0.264\text{--}0.283$) with no clear deprivation gradient, and the small “unknown” cohort ($n = 757$) sits highest at 0.313; on disability, non-disabled users (0.277, $n = 16,085$) marginally outperform disabled users (0.258, $n = 1,477$) — a 2-point gap. None of the gaps are large enough to indicate systematic exclusion, but the gender gap warrants flagging; it likely reflects course composition (STEM over-represented in the male sub-cohort) rather than a modelling choice, though I claim no causal knowledge. Reporting the audit satisfies the DEI objective; explaining *why* would require intervention data OULAD does not contain.

5.10 Hyperparameter Sensitivity

evaluation/tuning_{svd,als,hybrid}.csv record the full sweeps.

- **SVD** — $k \in \{20, 50, 100\}$: NDCG@10 differences small (0.159–0.167); $k = 100$ selected; plateau at $k > 50$ suggests effective rank saturated by 50 factors.
- **ALS** — 18-point grid ($n_{\text{factors}} \times \lambda \times \alpha$): best (32, 0.01, 1.0), NDCG@10 = 0.149. Increasing α *degraded* performance monotonically (0.149 at $\alpha = 1$ → 0.128 at $\alpha = 40$), contradicting the Hu-Koren-Volinsky music-recommendation setting — OULAD click distributions have heavy positive tails per user, so strong confidence overfits.
- **Hybrid** — 21-point simplex ($\alpha + \beta + \gamma = 1$, step 0.2); best (0.0, 0.8, 0.2). All $\alpha > 0.25$ configurations underperform CF-dropped ones, confirming the ablation.
- **Time-decay** — $\lambda \in \{0.0, 0.005, 0.01, 0.02\}$: $\lambda = 0.01$ produces the largest gain (+2.3 P@10 pts for Hybrid, +4.7 for Popularity); larger and smaller values regress toward raw clicks.
- **Robustness across seeds** — scripts/robustness_check.py with random_state $\in \{0, 1, 2\}$ (evaluation/robustness_summary.csv): Content, Hybrid, GatedHybrid, Popularity, SVD exactly deterministic; ALS drifts in the 4th decimal; Random shifts by 0.005 on HR@10. Model ordering identical across all seeds.

5.11 LambdaMART Re-Ranking Experiment

The two-stage LambdaMART reranker (Burges et al., 2010, §4.6): Stage 1 (tuned Hybrid) retrieves top-100; Stage 2 LightGBM LGBMRanker (objective='lambdarank') reorders via 35 (user, item) features. Honest three-window training design (Stage 1 refit on tune_train, labels from tune_val, evaluation on split.test) prevents test-window leakage. The baseline row in Table 5.5 is the *deployed* GatedHybrid — the strongest single-model system from §5.3 — so the comparison is “does the reranker beat what a marker would actually be served?”, not “does it beat its own Stage 1”.

Model	P@10	Recall@10	NDCG@10	HR@10	OWP@10
GatedHybrid (deployed baseline)	0.276	0.105	0.303	0.784	0.287
LambdaMART (rerank)	0.252	0.107	0.284	0.796	0.262

Table 5.5 — LambdaMART reranking versus the deployed GatedHybrid, primary split ($n_{\text{users}} = 17,562$).

The reranker **underperforms the deployed baseline on P/NDCG/OWP** (−2.3 / −1.9 / −2.6 pts) while marginally improving HR (+1.2) and Recall (+0.2) — it finds hits for more users but ranks them lower. Feature-importance (evaluation/reranker_importance.csv) surfaces activity_type=resource, log_access_count, mean_score_of_accessers, week_from_norm — precisely the item features the linear hybrid already exploits. Gradient-boosted trees add flexibility but that flexibility does not translate into ranking gains: content + outcome saturates achievable NDCG in the pointwise-linear space on OULAD’s course-scoped catalogue, and heavier LTR machinery does not close a gap that is essentially already closed.

5.12 Critical Evaluation and Limitations

The evaluation reaches the level of statistical confidence the marker specifically asked for: precise metric definition (Section 5.2), 95% CIs across ~17K users, paired testing with Bonferroni adjustment, five-fold temporal CV, ablation, cold-start stratification, fairness audit, and hyperparameter sensitivity across four axes plus a three-seed robustness sweep. The claim I make — that GatedHybrid is the strongest system on the primary split and across five folds — is supported at adjusted $p \leq 5 \times 10^{-5}$ on the four ranking-quality metrics against every baseline including Content (§5.5), with numerically-zero p -values against Random, Popularity, SVD, and ALS on precision, NDCG, and OWP.

Honest limits: OWP is *correlational*, not causal, so I cannot claim that issuing these recommendations would improve outcomes without an interventional study; ALS underperforms SVD and CF contributes negatively at every non-trivial weight, so the “collaborative filtering” contribution has to be reported honestly as adding little value on OULAD; the cold cohort ($n = 155$) is small, so the gated-hybrid cold-user precision carries a wider CI; the gender-fairness gap is descriptive, not diagnosed or mitigated (future work). None of these limitations undermine the primary claim — they are the honest boundaries of an offline observational evaluation on a single dataset.

6 Conclusion

6.1 Summary

This project set out to build and rigorously evaluate a hybrid recommendation engine combining collaborative filtering, content matching, and learning-outcome signals on OULAD, delivered as a reproducible pipeline under the CM3005 template. Seven recommenders — Random, Popularity, SVD, ALS, Content, Hybrid, GatedHybrid — plus a LambdaMART two-stage reranker sit behind a common interface. The evaluation reports six metrics on a primary temporal split with 95% bootstrap CIs, five-fold temporal cross-validation, paired- t with Bonferroni adjustment, ablation, cold-start stratification, a fairness audit, four-axis hyperparameter sensitivity, and a three-seed robustness sweep. The GatedHybrid is the leading model on Precision@10, NDCG@10, and OWP@10; the lead survives CV. The relative gain over the preliminary-report baseline is 18% on Precision@10. A FastAPI service, Streamlit dashboard, and two-stage Docker image round out the delivery.

6.2 Contribution

The contribution is not a new algorithm. It is a *systematic combination* of established techniques — collaborative filtering, content filtering, and outcome-aware evaluation — implemented and evaluated end-to-end on a public educational dataset with methodological rigour, plus a working serving stack and a LambdaMART reranker. Load-bearing pieces: (i) outcome-weighted Precision@K, precisely defined (§5.2), reducing to standard precision under a uniform-outcome null and computed strictly from training data; (ii) an eight-model progression exposing each component’s contribution — content dominates, outcome adds ~1 P@10 point, CF contributes nothing above content+outcome — supported by paired- t with Bonferroni on ~17K per-user observations (§5.5, adjusted $p \leq 5 \times 10^{-5}$ against Content and numerically zero against Random / Popularity / SVD / ALS on precision, NDCG, and OWP); (iii) the FastAPI + Streamlit + Docker stack that makes the recommendations *inspectable*, not just reported.

6.3 Broader Themes

Three themes deserve stating. The first is that ablation-driven design is worth doing explicitly. The Gated Hybrid did not appear in the preliminary design; it emerged from the cold-start stratification, itself added only because marker feedback asked for stratified reporting. What began as a hedge against a rubric criticism turned into the project’s best-performing recommender. Reporting the ablation honestly is what let the design decision surface — a pattern I will reuse.

The second theme is the tension between recommendation quality and popularity bias. Recommenders that maximise Precision@K on OULAD concentrate on the head of the item distribution — Gini 0.74 for the Gated Hybrid versus 0.32 for Random. Content-based cosine similarity drives this concentration; the outcome signal turns out to be essentially orthogonal to popularity (§5.8, $r \approx -0.08$), so it neither amplifies nor mitigates it. Head-concentration is not automatically bad for an educational recommender — course-central resources are course-central for a reason — but it is worth surfacing as a design trade-off: the choice of how to trade quality for diversity should sit with the pedagogical stakeholder, not with the recommender.

The third theme is that “canonical” methods do not transfer without empirical checking. Alternating Least Squares with confidence weighting is *the* implicit-feedback CF method (Hu, Koren & Volinsky, 2008), and I expected it to dominate SVD on this dataset. Instead it underperformed at every confidence-alpha value tried, because OULAD click distributions have heavy positive tails per user (browsing behaviour) that treating each click as strongly confident overfits. This is a finding that would not have surfaced without running the grid; it is also a caution against assuming that any method wins in a new domain simply because it wins in the paper’s original domain.

6.4 Limitations

Four limitations are worth restating. First, the outcome-weighted precision is a *correlational* proxy for learning benefit; without an interventional study I cannot claim that issuing these recommendations would improve outcomes. Second, the cold cohort ($n = 155$) is small, so the Gated Hybrid’s cold-user precision estimate carries a wider confidence interval than the aggregate. Third, the gender-fairness gap in Section 5.9 is descriptive: this project reports it but does not diagnose or mitigate it, though the score decomposition surfaced by the FastAPI /decompose endpoint gives a natural entry point for a fairness-aware re-ranking step (see Future Work). Fourth, the outcome signal `mean_score_of_accessors` is itself observational: it captures the score of learners who *chose* to access an item, so it is vulnerable in principle to self-selection — high-outcome items could reflect the cohort that accessed them rather than any property of the item. Empirically the concern turned out to be small (§5.8 reports near-zero correlation between the outcome signal and item popularity, and the `hybrid_no_outcome` ablation shifts Gini by only -0.017), but a causal cleanse of the outcome signal would require instrumented data OULAD does not provide.

6.5 Future Work

LambdaMART re-ranking — implemented as an eighth model (§4.6) and evaluated (§5.11) — did not beat the deployed GatedHybrid baseline on precision or NDCG at the current feature set; the linear content + outcome ensemble already saturates the achievable ranking quality on OULAD’s course-scoped catalogue. A natural extension is to enrich the 35 pointwise features with session-aware click sequences, cross-course popularity transfer, and per-user click-rate dynamics — signals outside my current feature builder that might give LambdaMART genuinely new information. A **fairness-aware re-ranking layer** would insert a post-hoc penalty on the hybrid score that trades a controlled precision loss for a demographic-parity or exposure-parity

improvement (Ekstrand et al., 2018; Zehlike et al., 2022); the score decomposition already returned by `/decompose` gives the mitigation step a direct hook into the deployed system. A grade-prediction diagnostic — correlating each model’s per-user precision with the eventual `final_result` in OULAD — would connect the offline OWP metric to a longitudinal outcome, tightening the “learning benefit” claim without an interventional study. A knowledge-tracing component (BKT or DKT) could enter as a fourth signal representing modelled learner state, letting the evaluation directly compare “knowledge-tracing-flavoured” and “content-and-outcome-flavoured” recommendations on the same footing. Beyond OULAD, cross-dataset generalisation to a MOOC log (MITx / HarvardX) would test whether the “content dominates, CF contributes negatively” finding is a property of OULAD’s course-scoped catalogue or a more general educational-recommender fact. Finally, the fairness gap warrants its own investigation with the specialist literature as its starting point — the numbers reported here are the input, not the conclusion, of that work.

References

1. Bousbahi, F. and Chorfi, H., 2015. MOOC-Rec: A case based recommender system for MOOCs. *Procedia — Social and Behavioral Sciences*, 195, pp.1813–1822.
2. Burges, C.J.C., 2010. From RankNet to LambdaRank to LambdaMART: An overview. *Microsoft Research Technical Report MSR-TR-2010-82*.
3. Burke, R., 2002. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), pp.331–370.
4. Corbett, A.T. and Anderson, J.R., 1995. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), pp.253–278.
5. Drachsler, H., Verbert, K., Santos, O.C. and Manouselis, N., 2015. Panorama of recommender systems to support learning. In: F. Ricci, L. Rokach and B. Shapira (eds.), *Recommender Systems Handbook*, 2nd edn., pp.421–451. Springer, New York.
6. Ekstrand, M.D., Tian, M., Kazi, M.R.I., Mehrpouyan, H. and Kluver, D., 2018. Exploring author gender in book rating and recommendation. *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*, pp.242–250.
7. Hlosta, M., Zdrahal, Z. and Zendulka, J., 2017. Ouroboros: Early identification of at-risk students without models based on legacy data. *Proceedings of the Seventh International Learning Analytics & Knowledge Conference (LAK ’17)*, pp.6–15.
8. Hu, Y., Koren, Y. and Volinsky, C., 2008. Collaborative filtering for implicit feedback datasets. *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM)*, pp.263–272.
9. Koren, Y., Bell, R. and Volinsky, C., 2009. Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8), pp.30–37.
10. Kuzilek, J., Hlosta, M. and Zdrahal, Z., 2017. Open University Learning Analytics dataset. *Scientific Data*, 4(1), pp.1–8.
11. Manouselis, N., Drachsler, H., Vuorikari, R., Hummel, H. and Koper, R., 2011. Recommender systems in technology-enhanced learning. In: F. Ricci, L. Rokach, B. Shapira and P.B. Kantor (eds.), *Recommender Systems Handbook*, 1st edn., pp.387–415. Springer, Boston.

12. Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L.J. and Sohl-Dickstein, J., 2015. Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28, pp.505–513.
13. Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L., 2012. BPR: Bayesian personalized ranking from implicit feedback. *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (UAI)*, pp.452–461.
14. Ricci, F., Rokach, L. and Shapira, B. (eds.), 2015. *Recommender Systems Handbook*, 2nd edn. Springer, New York.
15. Rienties, B., Boroowa, A., Cross, S., Kubiak, C., Mayles, K. and Murphy, S., 2016. Analytics4Action Evaluation Framework: A review of evidence-based learning analytics interventions at the Open University UK. *Journal of Interactive Media in Education*, 2016(1), Article 2, pp.1–11.
16. Thai-Nghe, N., Drumond, L., Krohn-Grimberghe, A. and Schmidt-Thieme, L., 2010. Recommender system for predicting student performance. *Procedia Computer Science*, 1(2), pp.2811–2819.
17. Wilson, K.H., Karklin, Y., Han, B. and Ekanadham, C., 2016. Back to the basics: Bayesian extensions of IRT outperform neural networks for proficiency estimation. *Proceedings of the 9th International Conference on Educational Data Mining*, pp.539–544. arXiv:1604.02336.
18. Zawacki-Richter, O., Marín, V.I., Bond, M. and Gouverneur, F., 2019. Systematic review of research on artificial intelligence applications in higher education — where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1), Article 39, pp.1–27.
19. Zehlike, M., Yang, K. and Stoyanovich, J., 2022. Fairness in ranking, Part I: Score-based ranking. *ACM Computing Surveys*, 55(6), Article 118, pp.1–36.
20. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y., 2017. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, pp.3146–3154.
21. Frederickson, B., 2018. implicit: Fast Python collaborative filtering for implicit datasets. Version 0.7. GitHub: benfred/implicit.
22. Herlocker, J.L., Konstan, J.A., Terveen, L.G. and Riedl, J.T., 2004. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1), pp.5–53.
23. Adomavicius, G. and Tuzhilin, A., 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), pp.734–749.
24. Yudelson, M.V., Koedinger, K.R. and Gordon, G.J., 2013. Individualized Bayesian knowledge tracing models. In: *Proceedings of the 16th International Conference on Artificial Intelligence in Education (AIED)*, pp.171–180.
25. Baker, R.S. and Inventado, P.S., 2014. Educational data mining and learning analytics. In: *Learning Analytics: From Research to Practice*. Springer, New York, pp.61–75.
26. Efron, B. and Tibshirani, R.J., 1993. *An Introduction to the Bootstrap*. Chapman & Hall, New York.

27. Ramírez-Correa, P.E., Rondán-Cataluña, F.J., Arenas-Gaitán, J. and Alfaro-Pérez, J.L., 2017. Moderating effect of learning styles on a learning management system's success. *Telematics and Informatics*, 34(1), pp.272–286.